

De API's van een betrouwbare bron

Project Uit Betrouwbare Bron (UBB)

Werkgroep Historie – Kennisplatform API's
2 september 2026



Naar schatting tienduizenden burgers per jaar

Levensontwrichtend



- Kinderopvangtoeslagaffaire
- Schadeafhandeling aardgaswinning
- Fraude Signalering Voorziening
- WIA-uitkeringen
- Boodschappenaffaire
- Recipients weigeren wanbetalers in de cel te stoppen

Het Parool

Opinie Kunst & Media Nieuws Plus

Al 18 jaar in: de con

53.000 de groen licht

Het UWV weet al uitkeringen, maar plan voor alweer een hersteloperatie ligt op de plank, en daarvoor moeten tienduizenden dossiers tegen het licht

Wet en regelgeving

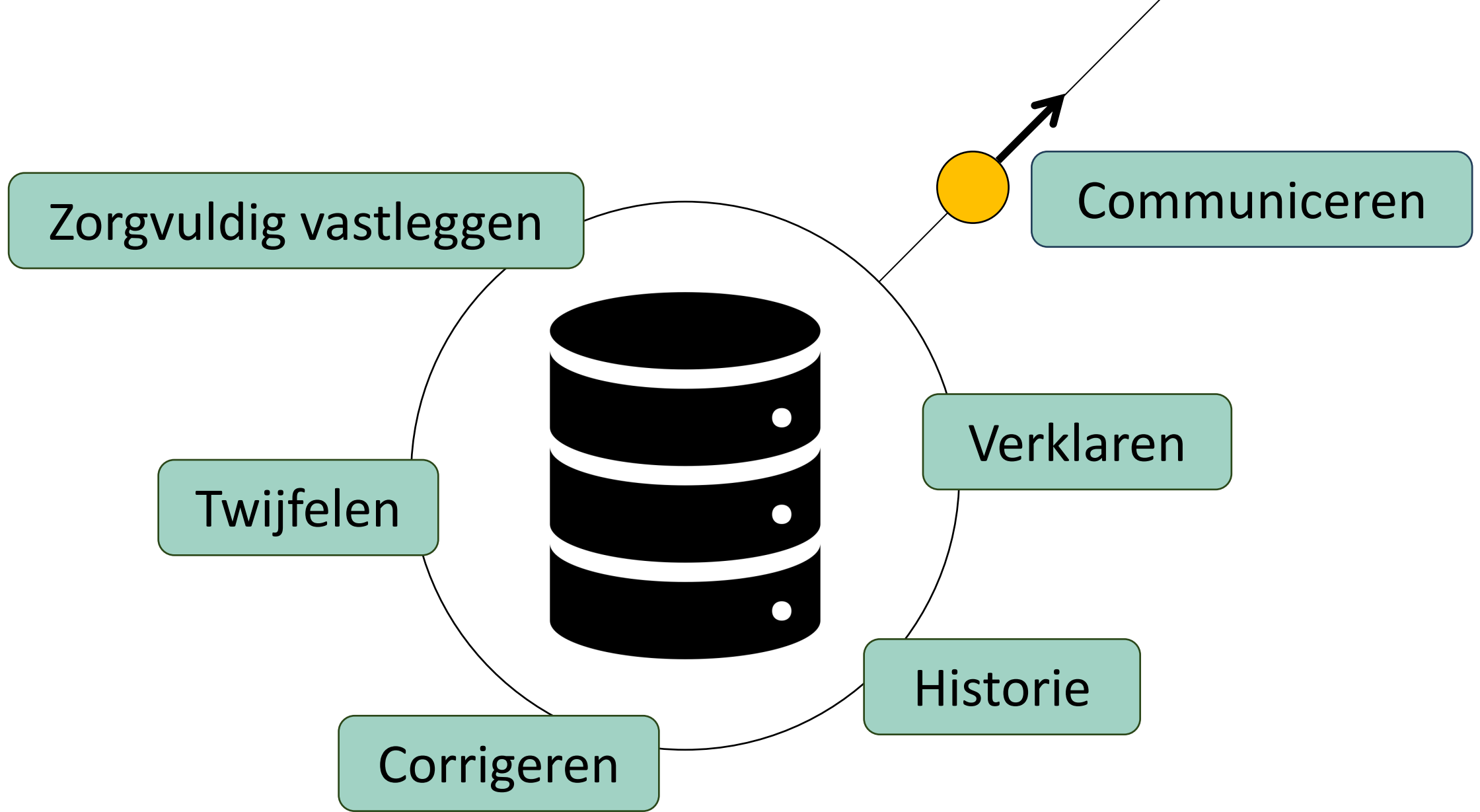
Organisatie 'ecosysteem'

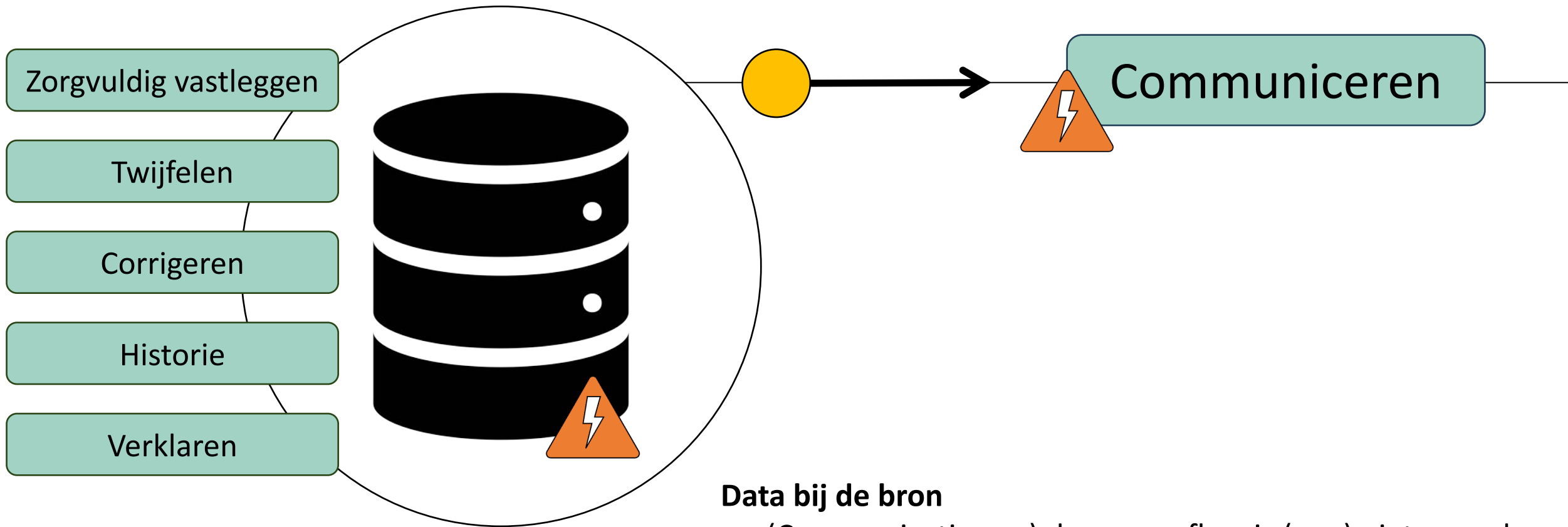
Betekenis

Techniek

Dienstverlening







Data bij de bron

- (Communicatie van) de crappy flow is (nog) niet op orde
- Dus afnemers krijgen niet alle informatie
- 100% zeker dat de kopie niet betrouwbaar is
- Dus **niet** kopiëren !!

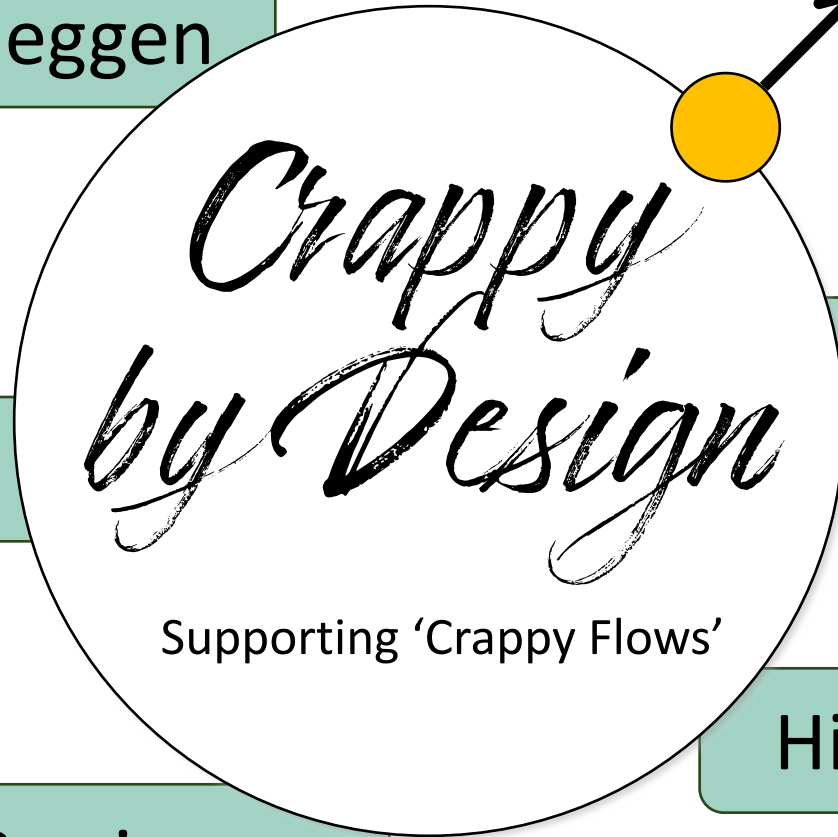
Paradoxaal

- (Communicatie van) de crappy flow op orde?
Dan is kopiëren minder erg! (managed copy)

Zorgvuldig vastleggen

Twijfelen

Corrigeren

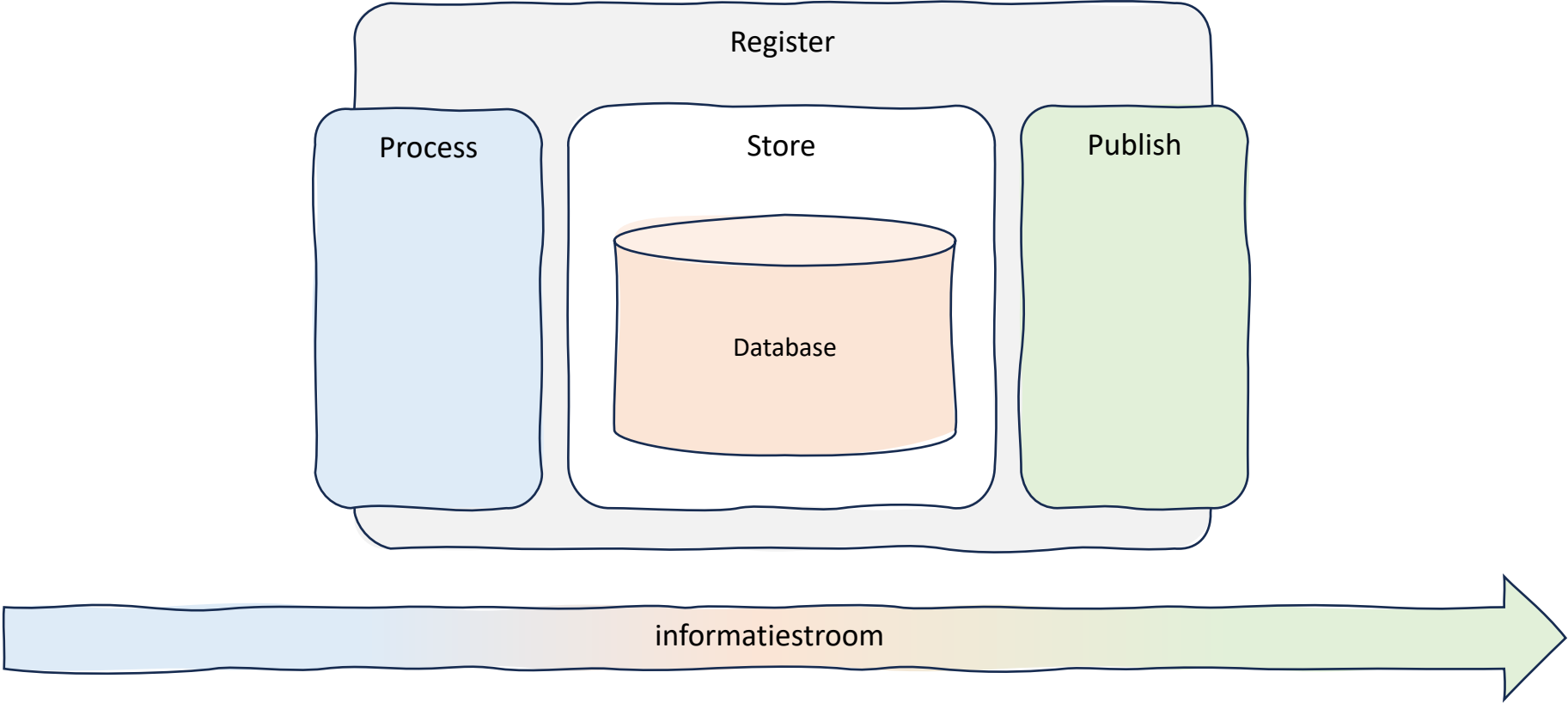


Verklaren

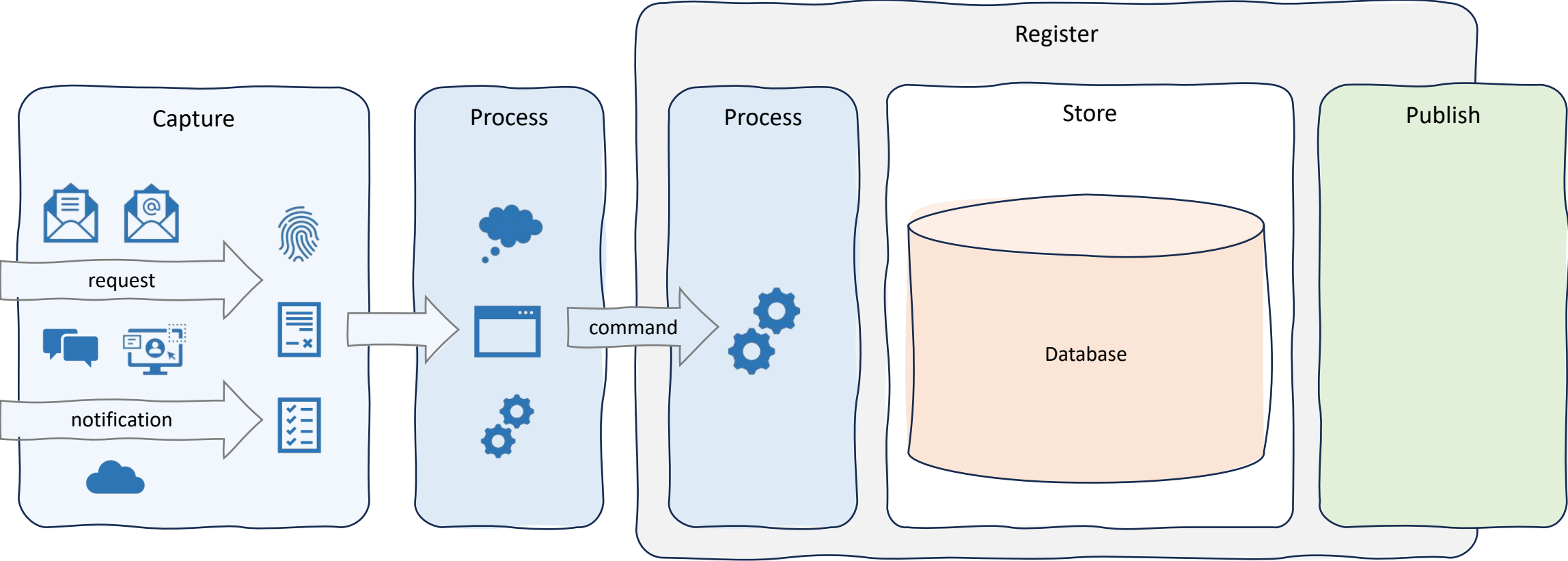
Historie

Communiceren

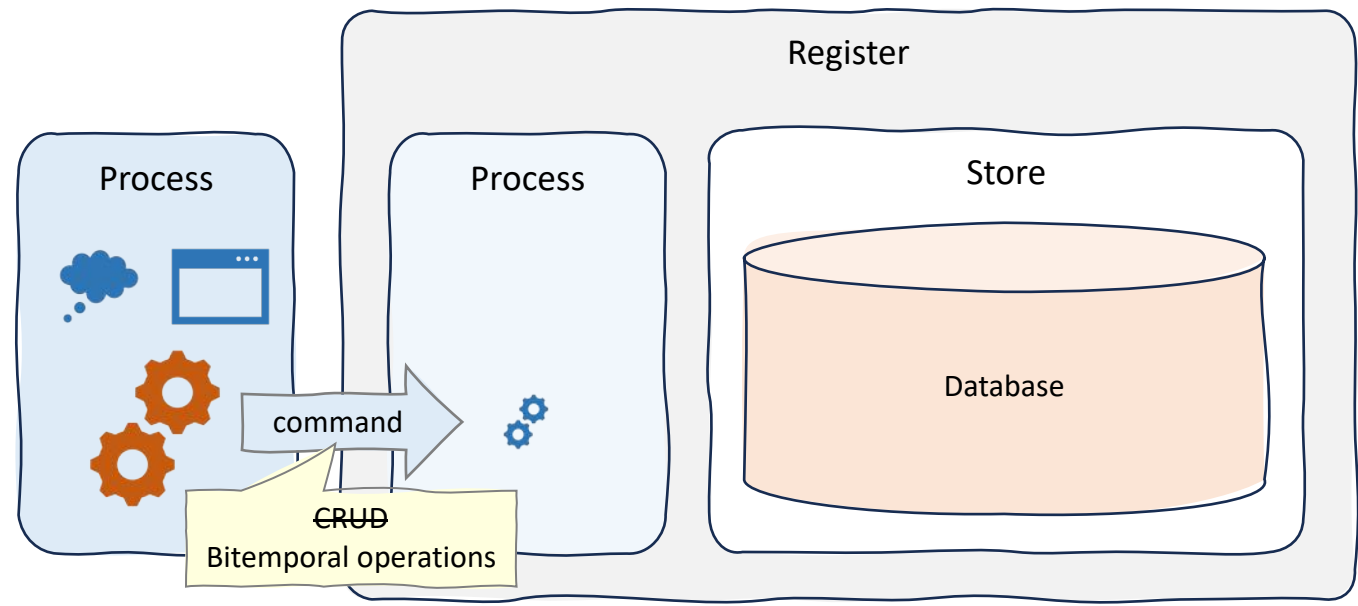
Register



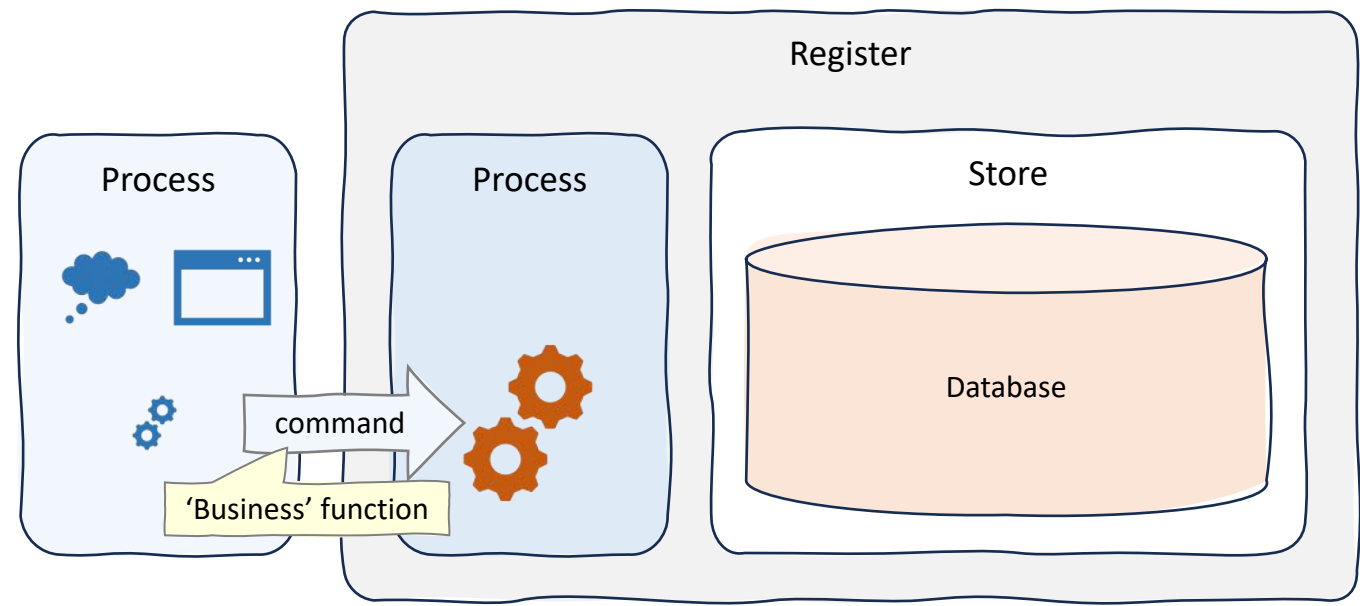
Domain



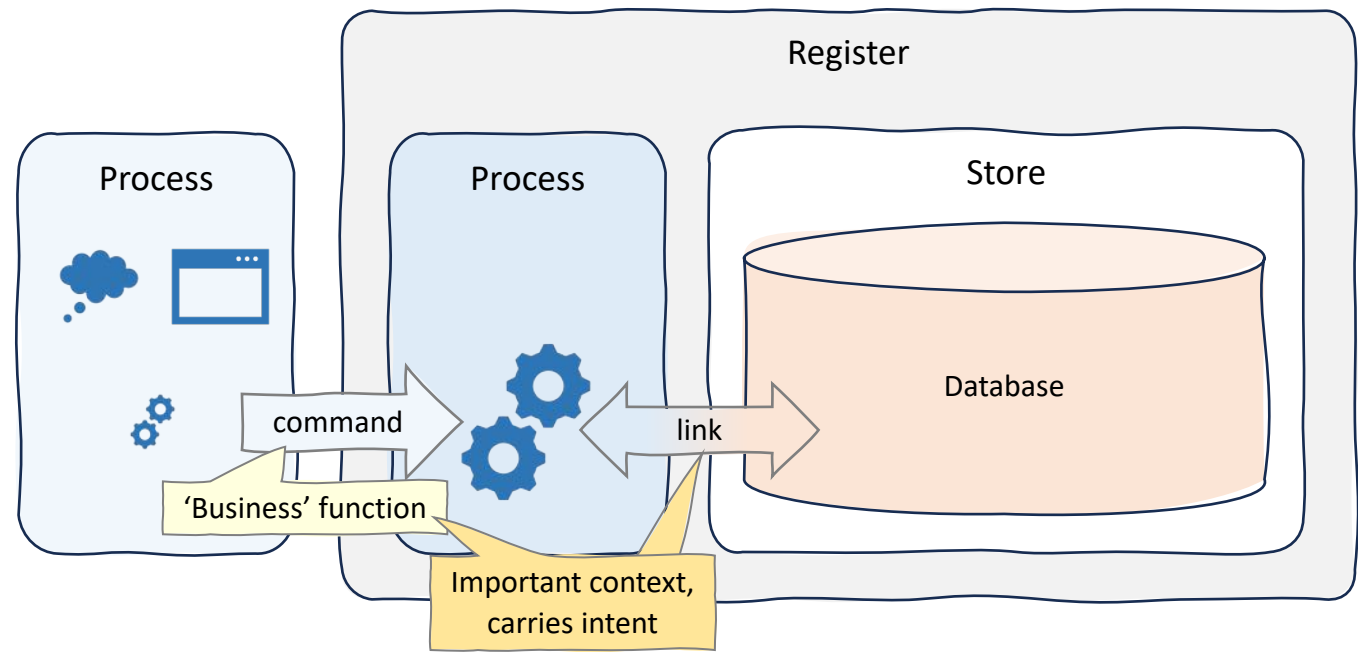
Commands ?



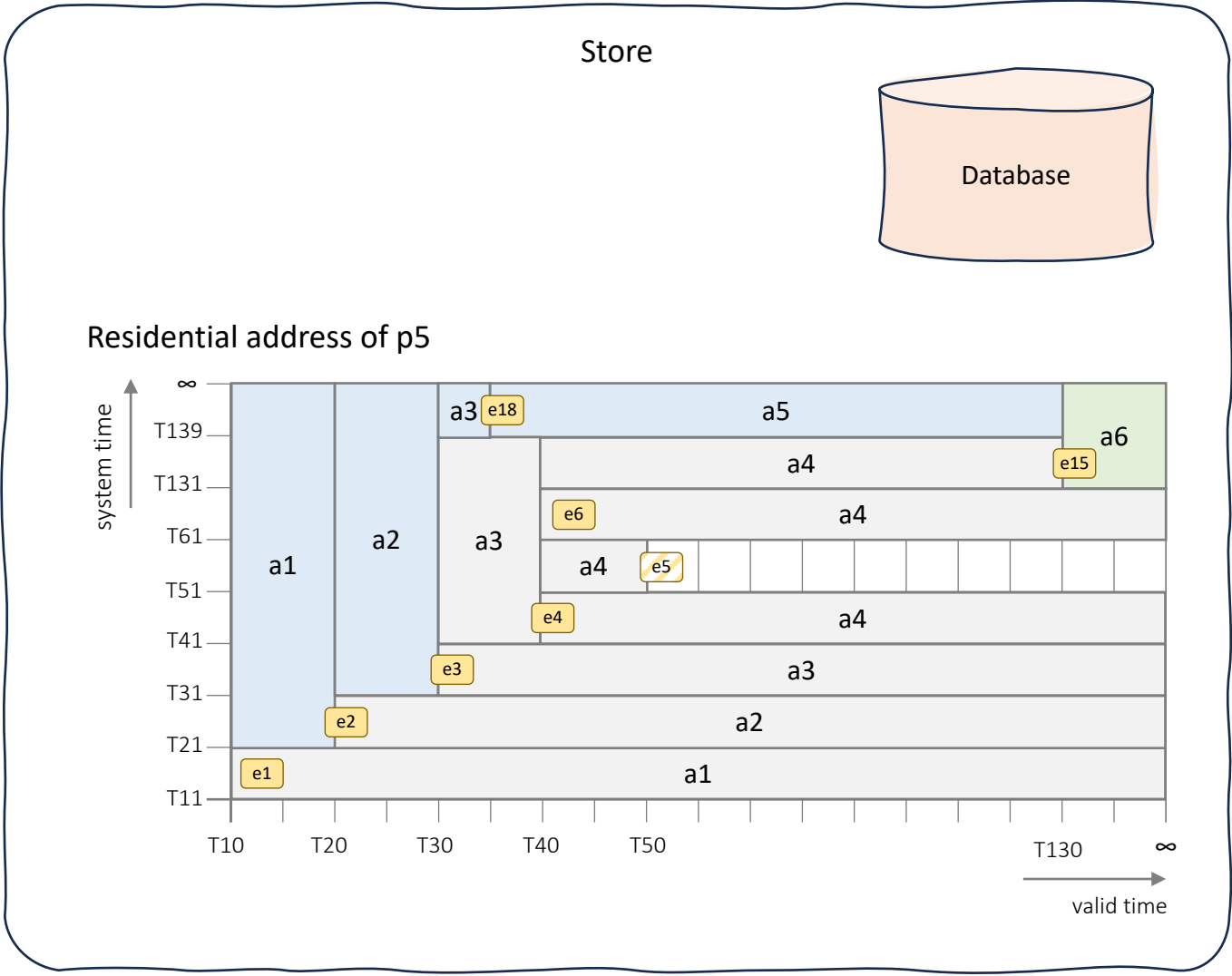
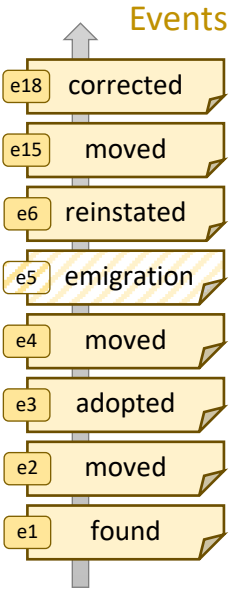
Commands !



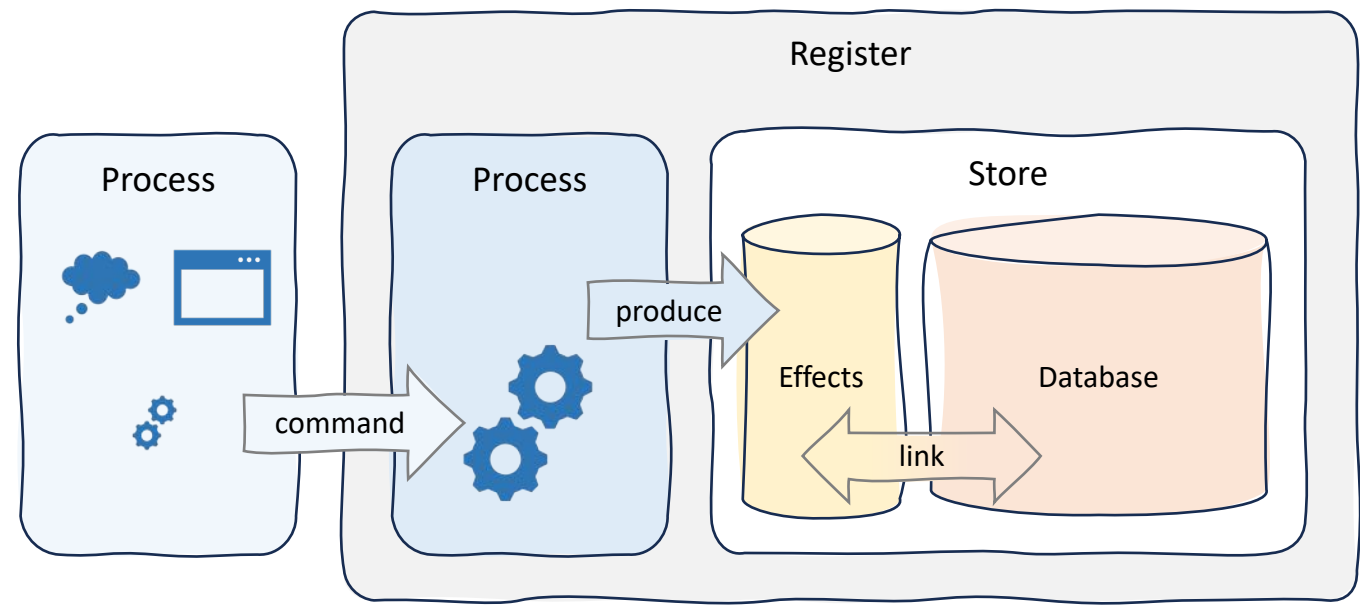
Provenance ?



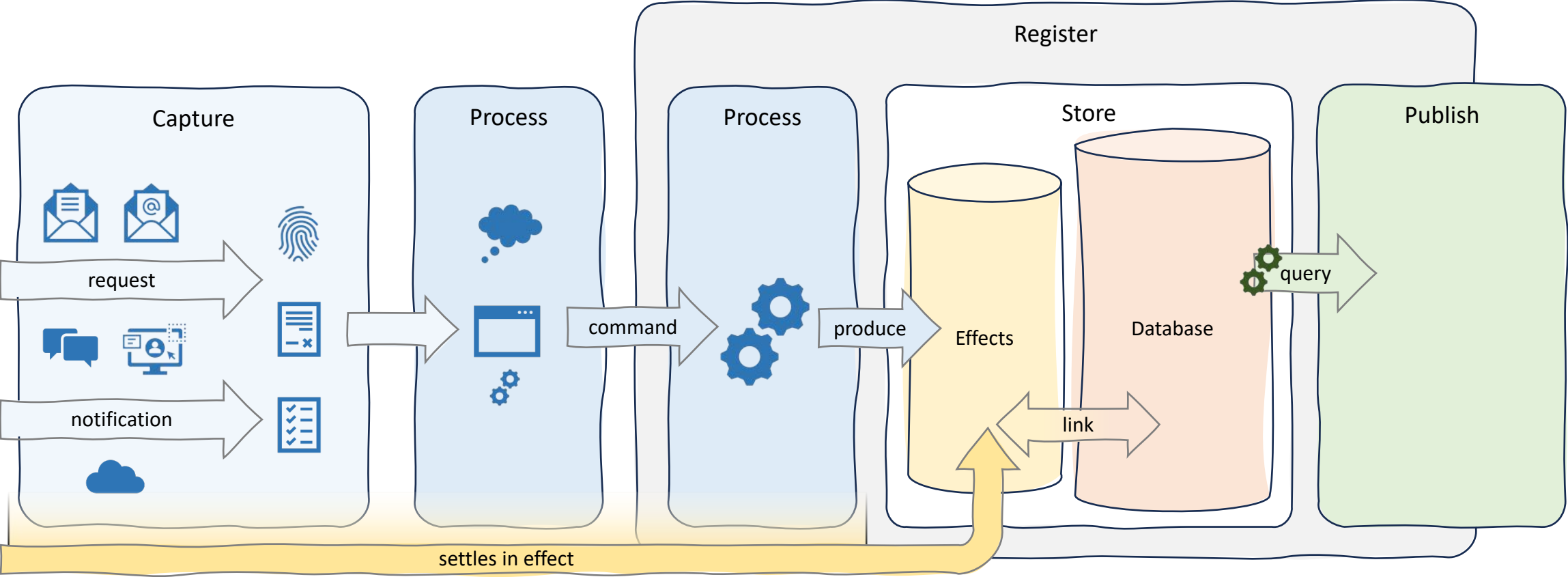
Possible store design



Effect = 'Event' in Event Sourcing



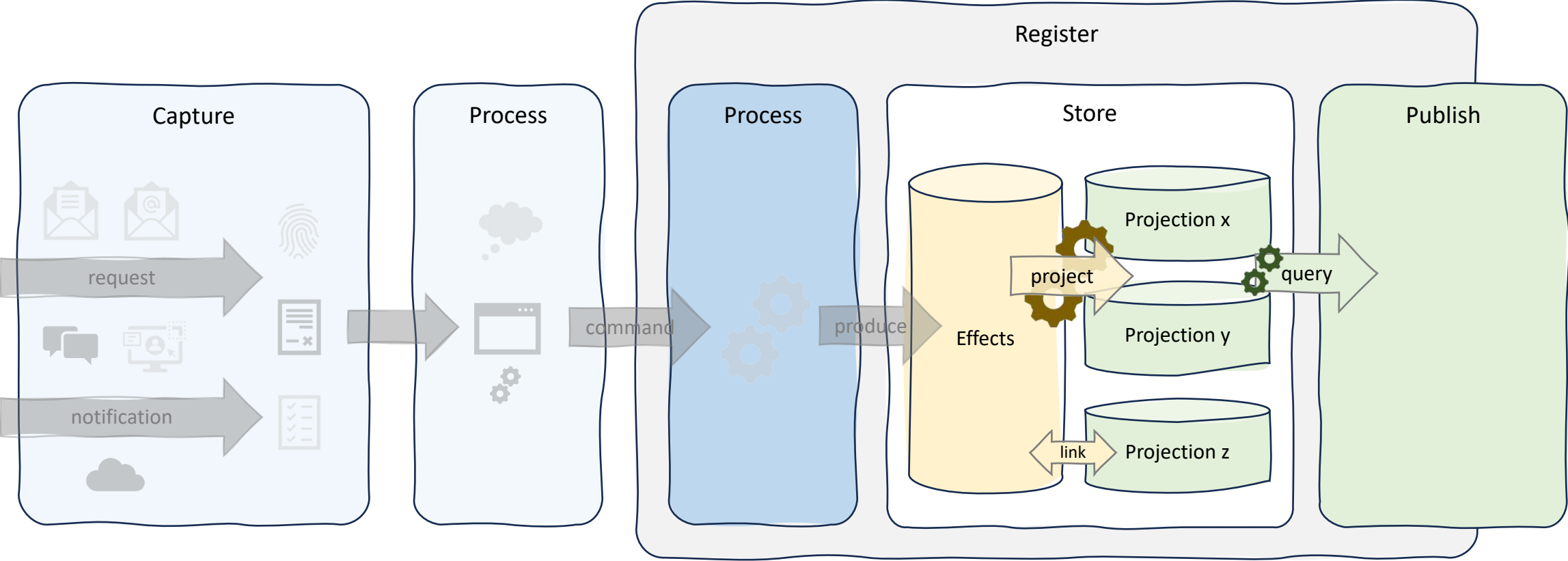
Effects as part of the provenance



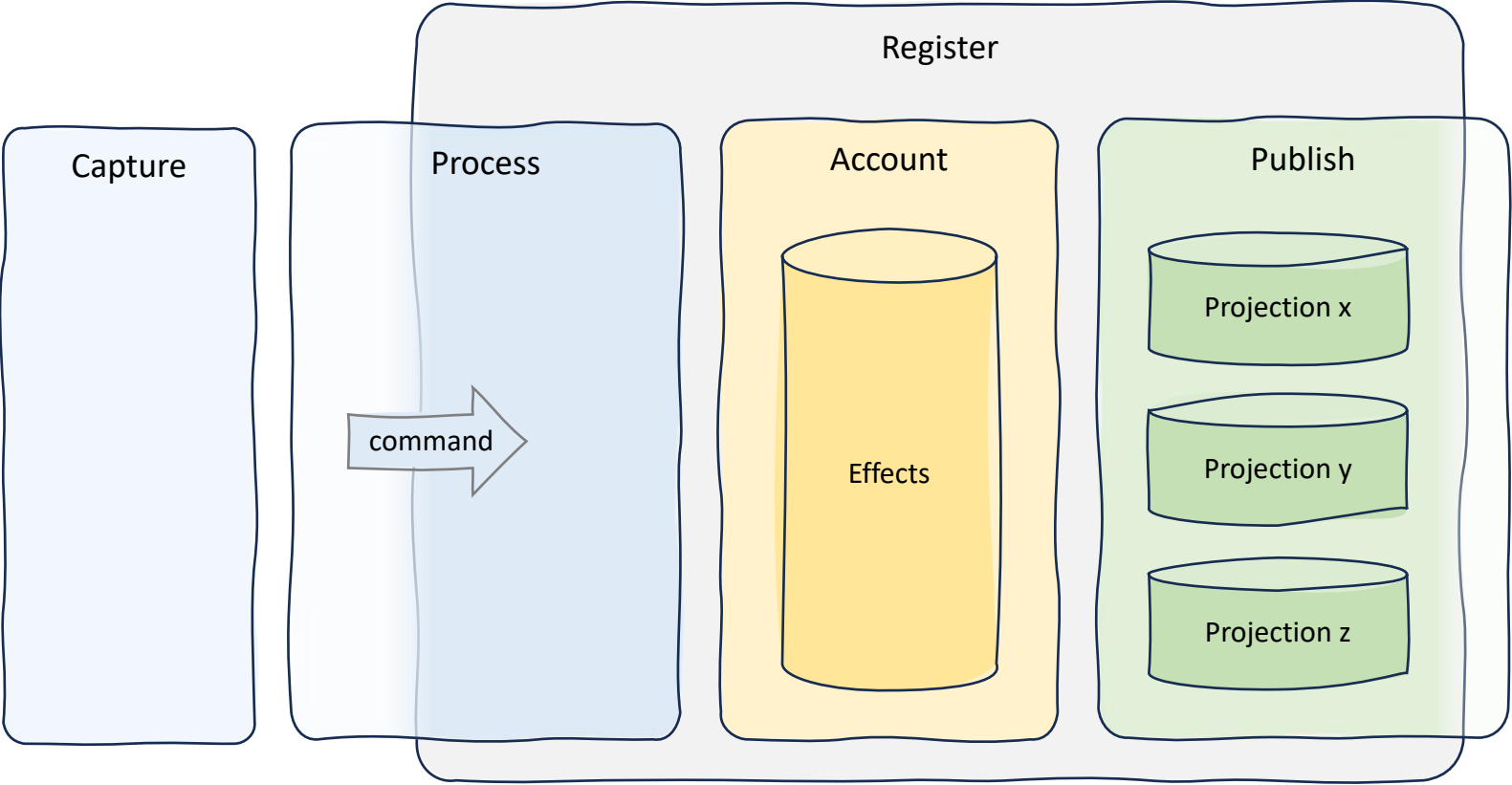
Possible store design



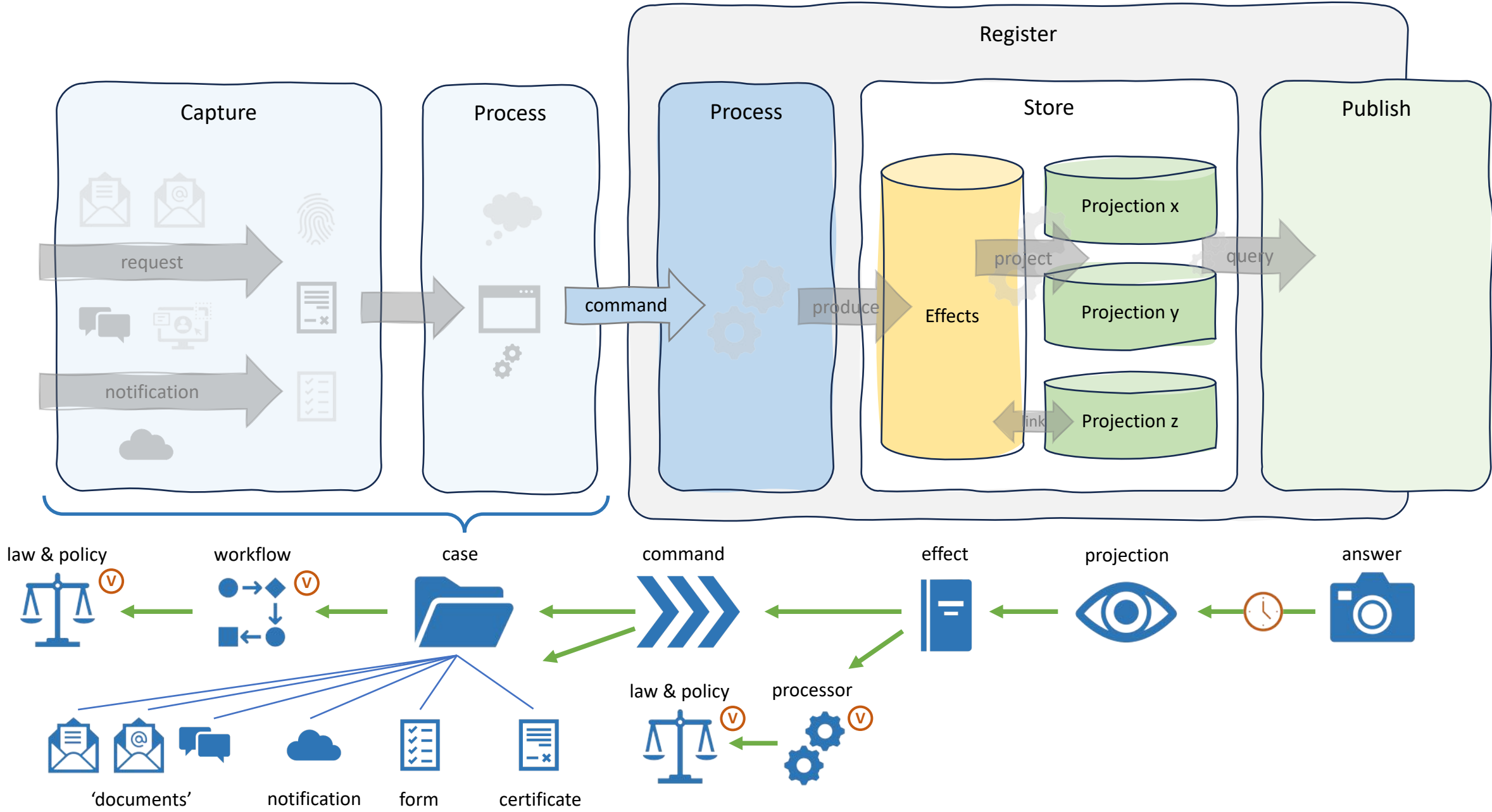
Projections



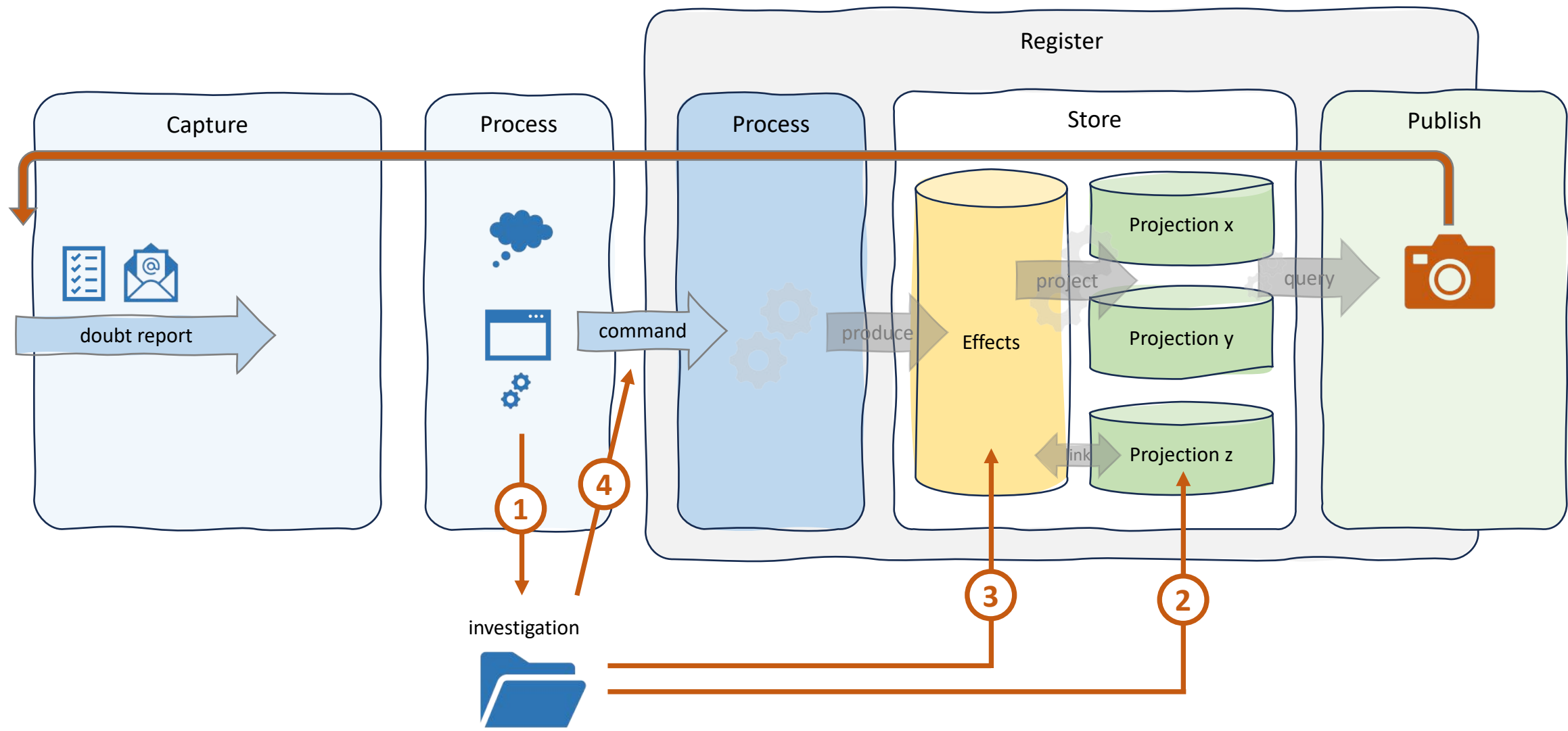
Register '2.0'



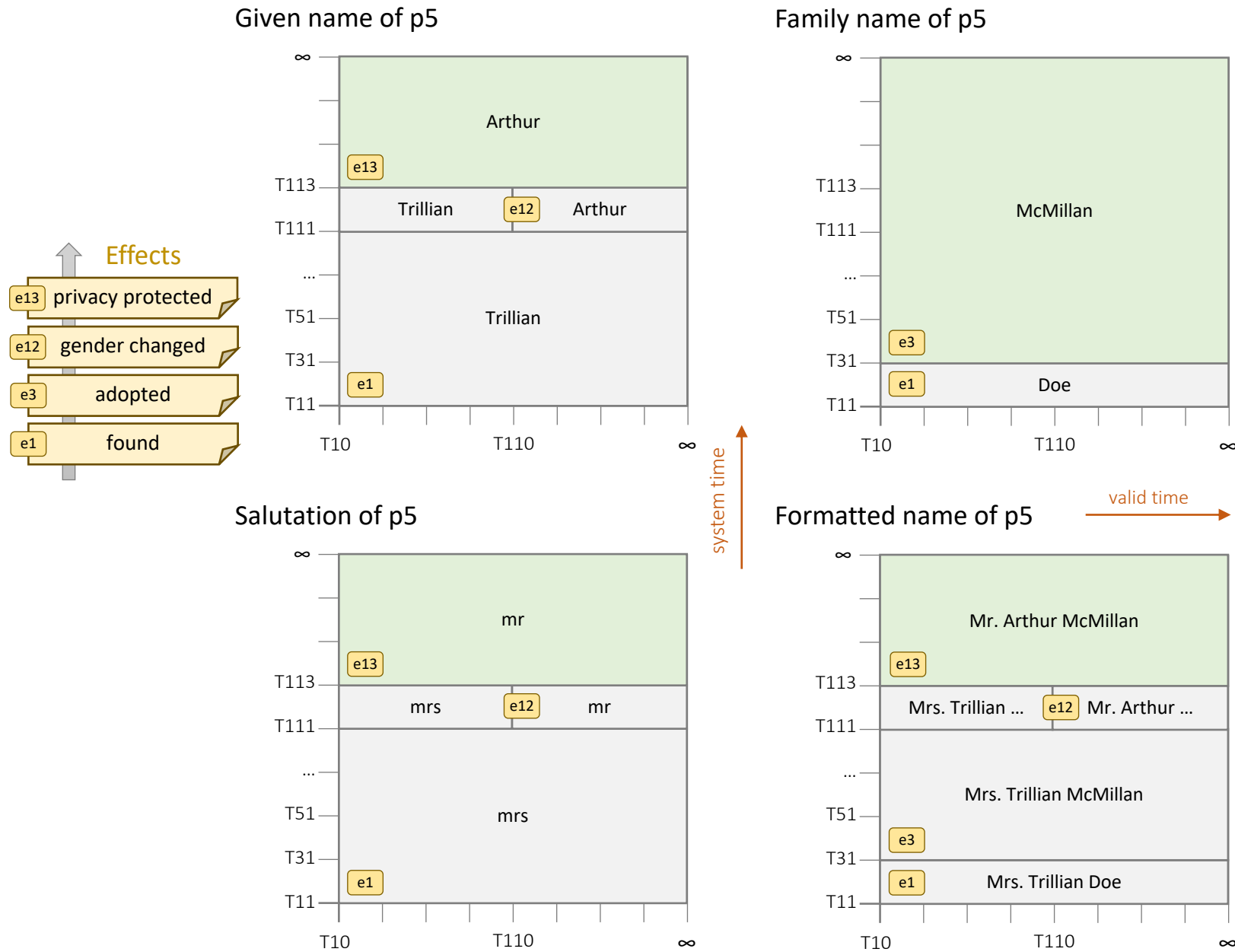
Provenance

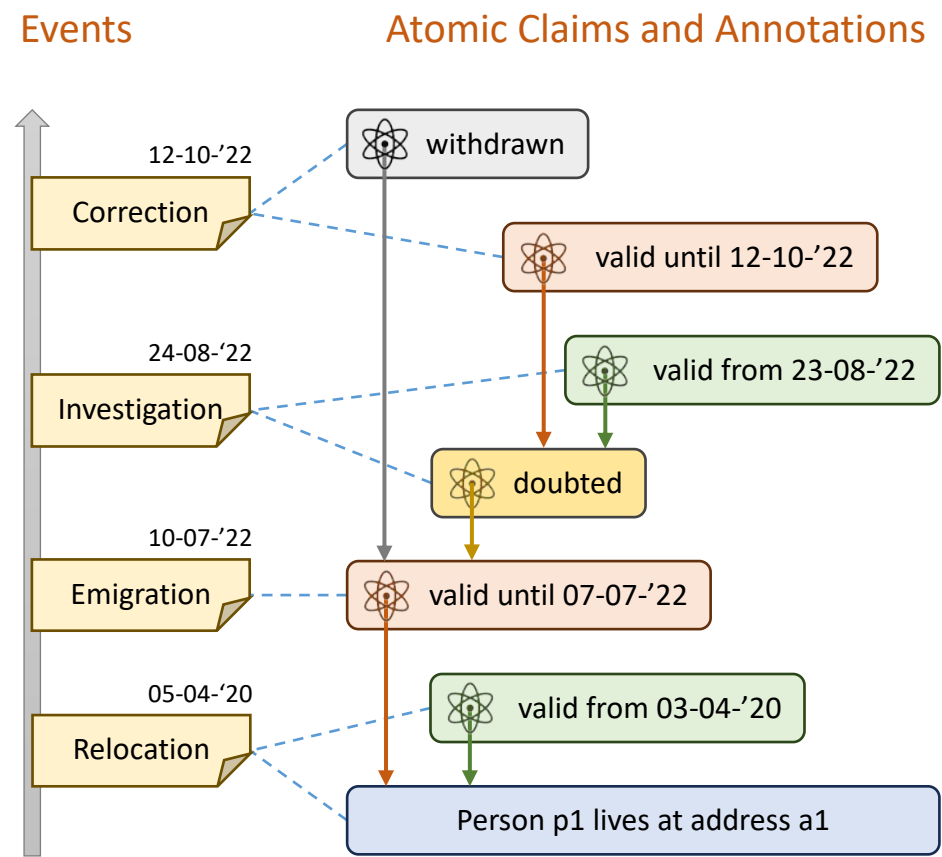


Doubt



Sixth normal form





PhD Thesis:
Towards epistemically humble
government registers

8 APIs (STUB)

This chapter is eventually going to be filled by the historie werkgroep of the NL API Strategie. Below is a short inventory of potential topics.

8.1 Commands

8.1.1 Functional APIs

- What shape do functional APIs take, REST, GraphQL, something else? And what shape within REST or GraphQL?
- How do we include the capture side? How do we link to cases and artefacts in a DMS?
- The information from the capture side might not apply to all the information in a command. Some commands are complex and have several subsections that must be processed as one business transaction yet potentially carry different provenance and might come into effect at different times.
- We may have to include references to law and legislation - as part of the provenance - for those situations where the default does not suffice (the nationality example from chapter 6)

8.1.2 Conversational APIs - The impact of doubt

- We want to defer as much processing as we can to the register. Certainly where there are multiple clients. But what if we have rules or an algorithm that encounters doubt? It is practically impossible to design algorithms that foresee every situation. So we need a generic mechanism to deal with doubt. In practice it's feasible to detect if a rule used data that was under investigation. When that happens the severity of the rule should be lowered. Even a hard rule might become one that can be overruled. The decision to overrule lies with the civil servant, not with the register. But that means that the register and the civil servant need to be able to communicate about this. It's not just a command, but a back and forth.

8.1.3 Business transactions - The impact of corrections

- We argued in earlier chapters that we cannot know all corrections beforehand. So we need a mechanism to deal with that. Several solutions are possible.

saction. They should be handled as a single operation in order to avoid catastrophic consequences if this is omitted (notifications will fire on catastrophic consequences).

ions

T professional - found the store chapter (chapter 4) and imagine how this is for a domain expert. So we might describe in the previous paragraph) and see how it plays out in a functional API.

s (chapter 8) on the inside and for some registers

el. So if a consumer asks a question, how do we answer it in a few milliseconds? The practical answer: find the most relevant data, which is rather hard in practice because we don't have a single source of truth (the concurrency subject). One thing is sure: to be able to turn the answer that was in the store at T80 into a new answer.

umer: "We answered your question at T80." This is the moment of change.

solve an authorisation problem.



The 'Command' side

Functional APIs

- What shape do functional APIs take, REST, GraphQL, something else? And what shape within REST or GraphQL?
- How do we include the capture side? How do we link to cases and artefacts in a DMS?
- The information from the capture side might not apply to all the information in a command. Some commands are complex and have several subsections that must be processed as one business transaction yet potentially carry different provenance and might come into effect at different times.
- We may have to include references to law and legislation - as part of the provenance - for those situations where the default does not suffice (the nationality example from chapter 6)

Conversational APIs - The impact of doubt

- We want to defer as much processing as we can to the register. Certainly where there are multiple clients. But what if we have rules or an algorithm that encounters doubt?
- It is practically impossible to design algorithms that foresee every situation. So we need a generic mechanism to deal with doubt.
- In practice it's feasible to detect if a rule used data that was under investigation. When that happens the severity of the rule should be lowered. Even a hard rule might become one that can be overruled. The decision to overrule lies with the civil servant, not with the register.
- But that means that the register and the civil servant need to be able to communicate about this. It's not just a command, but a back and forth.

Business transactions - The impact of corrections

- We argued in earlier chapters that we cannot know all corrections beforehand. So we need a mechanism to deal with that. Several solutions have been proposed (and some built):
- If you have a DevOps setting and the law allows you enough processing time, you can build new commands and effects just in time.
- If your register uses functional decomposition on the inside (an adoption is a change of parents, potentially a change of family name, a change of address etc.), you might expose those smaller (decomposed) commands to the outside, perhaps only as part of a wrapping correction command.
- You could do the same with effects: you compose your command of the effects that you would have liked to have.
- You could resort to offering direct access to bitemporal operations, only within a correction wrapper.
- All corrections must take place as a business transaction. They should be handled as a single operation in processing and in publication. Consequences can be severe if this is omitted (notifications will fire on temporarily inconsistent states with potentially catastrophic consequences).

Dry-run APIs - The impact of corrections

Correcting the past can be daunting. If you - as an IT professional - found the store chapter (chapter 4) hard - you are in good company around the world. Imagine how this is for a domain expert. So we might need dry runs to slowly build the correction (as described in the previous paragraph) and see how it plays out in a projection. This is another type of conversational API.

Bitemporal operations

We might want APIs that offer bitemporal operations (chapter 8) on the inside and for some registers perhaps also on the outside (or for the corrections).

The 'Publication' side

Repeatable question

- Systems normally run multiple write requests in parallel. So if a consumer asks a question, how do we know that what we have just read won't change in a few milliseconds? The practical answer: find the most recent moment where no writers were active. Sounds doable, is rather hard in practice because we don't know the commit times of transactions. (see chapter 12, the concurrency subject). One thing is sure: to be repeatable, a question asked at T100 might very well return the answer that was in the store at T80 because no writers that started before T80 are still active.
- That means we need to return this moment to the consumer: "We answered your question at T80."
- But that might expose unwanted information: The last moment of change.
- So perhaps we should return a token.
- If that token is specific to the consumer, that might also solve an authorisation problem that we have. Lawmakers want the impossible: accountability and a real erase. We can't do both so we offer a soft delete. That soft delete should still return what once was for repeatable questions (we think, up for debate). If it should, we can limit such questions to those who actually asked them before, because the token is the proof that they did.

Information products

If we really embrace projections, we can come closer to the needs of consumers. Where we now expose (rather than publish) databases, we can expose information in denormalised form, etc. Perhaps other types of APIs like GraphQL are more suitable for such APIs.

Notifications

Notifications are projections too - by design. A lot of consumers are not allowed to be notified about adoptions, but they are allowed to be informed about a changed family name or a changed address.